WEB ENGINEERING

Introduction to JavaScript

JavaScript



- Client Side Language (Vanilla JS, jQuery, React, D3.js, Vue.js etc)
- Server Side Language (Node.js, Express, MongoDB etc)
- Easy to Learn
- ES6 (ECMAScript 6) introduced and standardized in 2015

JS Syntax



- Very similar to other programming languages
- JavaScript uses the **`var`** keyword to declare variables.
- Universal principle for identifiers
- **Strings** are text, written within double or single quotes or back tick `
- Output on console using **`console.log`**

```
var x, y, z;        // How to declare variables
x = 5; y = 6;       // How to assign values
z = x + y;          // How to compute values
```

```
"John Doe"

'John Doe'
```

JS Variables



- Can create uninitialized variables or variable declaration (undefined)
- Change type of same variable anytime
- Recreate same variable again

```
var a; //undefined
var b = a + 10; // NaN
var x = 10
var x = 100
console.log(x) // 100
var x = "I am one hundred"
console.log(x) // I am one hundred
```

JS Operators



- Similar to other programming languages except === and !==

```
var a = 10
var b = '10'

console.log("a == b:", a == b)
console.log("a === b:", a === b)
```

```
a == b: true
a === b: false
```

Operator	Description
==	equal to
===	equal value and equal type
!=	not equal
!==	not equal value or not equal type
>	greater than
<	less than
>=	greater than or equal to
<=	less than or equal to
?	ternary operator

JS DataTypes



- Number, String, Object etc
- Decimal

```
var x = 15.56
```

- Exponential

```
var x = 10e5
```

- Object

```
var person = { firstName:"John", lastName:"Doe", age:50 };
```

JS typeof Operator



- You can use the JavaScript typeof operator to find the type of a JavaScript variable.
- The typeof operator returns the type of a variable or an expression:

```
typeof ""              // Returns "string"
typeof "John"          // Returns "string"
typeof "John Doe"      // Returns "string"
```

```
typeof 0               // Returns "number"
typeof 314             // Returns "number"
typeof 3.14            // Returns "number"
typeof (3)             // Returns "number"
typeof (3 + 4)         // Returns "number"
```

JS undefined vs null



- In JavaScript null is "nothing". It is supposed to be something that doesn't exist.
  - Unfortunately, in JavaScript, the data type of null is an object.
  - You can empty an object by setting it to null
- undefined and null are equal in value but different in type:

```
typeof undefined               // undefined
typeof null                    // object

null === undefined             // false
null == undefined              // true
```

Primitive Data



- A primitive data value is a single simple data value with no additional properties and methods.
- The `typeof` operator can return one of these primitive types:
  - `string`
  - `number`
  - `boolean`
  - `undefined`

```
typeof "John"                // Returns "string"
typeof 3.14                  // Returns "number"
typeof true                  // Returns "boolean"
typeof false                 // Returns "boolean"
typeof x                     // Returns "undefined" (if x has no value)
```

Complex Data



- The `typeof` operator can return one of two complex types:
  - `function`
  - `object`
- The `typeof` operator returns "object" for objects, arrays, and null.
- The `typeof` operator does not return "object" for functions.

```
typeof {name:'John', age:34} // Returns "object"
typeof [1,2,3,4]             // Returns "object" (arrays are objects)
typeof null                  // Returns "object"
typeof function myFunc(){}   // Returns "function"
```

ES6 Variables



	`var`	`let`	`const`
Scope	Function	Block	Block
Can change value after creation?	Yes	Yes	No

JS Functions

- Block of code designed to perform a particular task.

```
function myFunction(p1, p2) {
  return p1 * p2;   // The function returns the product of p1 and p2
}
```

- A JavaScript function is defined with the `function` keyword, followed by a name, followed by parentheses ().
- Function arguments are the values received by the function when it is invoked.
- Inside the function, the arguments (the parameters) behave as local variables

JS Functions – What will be the output? 1/2



1

```
function func() { return "Function 1"; }
function func() { return "Function 2"; }
console.log(func());
```

2

```
function func(a) { return "Function 1"; }
function func(a) { return "Function 2"; }
console.log(func());
```

3

```
function func(a) { return "Function 1"; }
function func(a, b) { return "Function 2"; }
console.log(func());
```

4

```
function func(a) { return "Function 1"; }
function func(a, b) { return "Function 2"; }
console.log(func(5));
```

5

```
function func(a) { return "Function 1"; }
function func(a, b) { return "Function 2"; }
console.log(func(5, 6));
```

JS Functions – What will be the output? 2/2



1

```
function func() { return "Function 1"; }
function func2() { return "Function 2"; }
console.log(func());
```

2

```
function func(a) { return "Function 1"; }
function func2(a) { return "Function 2"; }
console.log(func2());
```

3

```
function func(a) { return "Function 1"; }
function func2(a, b) { return "Function 2"; }
console.log(func2(5));
```

4

```
function func(a) { return "Function 1"; }
function func2(a, b) { return "Function 2"; }
console.log(func2(5,10));
```

5

```
function func(a) { return "Function 1"; }
function func(a, b) { return "Function 2"; }
console.log(func(5));
```

JS Functions – Check for **undefined**

```
function multiply(a, b) {
    a = typeof a !== 'undefined’ ? a : 0;
    b = typeof b !== 'undefined' ? b : 1;
    return a * b;
}

multiply(5); // 5
```

- General strategy for setting defaults is to test parameter values in the body of the function and assign a value if they are undefined or null

JS Functions – ES6 Default Parameters

```
function multiply(a = 0, b = 1) {
    return a * b;
}
multiply(); // 0
multiply(5); // 5
multiply(5, 5); // 25
```

- With default parameters, a manual check in the function body is no longer necessary

JS Functions – Arguments Object

- The arguments of a function are maintained in an array-like object
- All arguments can be retrieved using `arguments` object

```
function myConcat(separator) {
    var result = ''; // initialize list
    // iterate through arguments
    for (let i = 1; i < arguments.length; i++)
        result += arguments[i] + separator;
    return result;
}
// returns "red, orange, blue, "
var result = myConcat(', ', 'red', 'orange', 'blue');
// returns "elephant; giraffe; lion; cheetah; "
var result = myConcat('; ', 'elephant', 'giraffe', 'lion', 'cheetah');
// returns "sage. basil. oregano. pepper. parsley. "
var result = myConcat('. ', 'sage', 'basil', 'oregano', 'pepper', 'parsley');
```

Note: The `arguments` variable is "array-like", but not an array. It is array-like in that it has a numbered index and a `length` property. However, it does *not* possess all of the array-manipulation methods.

JS Functions – Rest Parameters

- The rest parameter syntax allows us to represent an indefinite number of arguments as an array.
- Rest Parameters are received as an Array and all Array operations can be applied

```
function fun(arg1, ...args) {
    return args.length + 1;
}
console.log(fun(0, 1, 2, 3, 4, 5, 6, 7, 8, 9) + " arguments passed");
// 10 arguments passed
```

```
function sumNums(...nums) {
    let sum = 0;
    for (let i = 0; i < nums.length; i++)
        sum += nums[i];
    return sum;
}
console.log(sumNums(1, 1, 1, 1, 1));
```

JS Functions – Pass by Value

- JavaScript is Pass by Value for Primitive Data (Number, String, Boolean)

```
function sum(n1, n2) {
    n1 = n1 + n2;
    return n1;
}
var a = 10, b = 10
console.log(a, "+", b, "=", sum(a, b))
```

```
function strFun(p1) {
    p1 = "Eleven"
}
var a = "Ten"
strFun(a)
console.log(a)
```

JS Functions – Pass by Reference

- Objects are of Reference Type
  - Therefore, JavaScript is Pass by Reference for Objects

```
function strFun(p1) {
    p1.value = "Eleven"
}
var a = { value: "Ten" }
strFun(a)
console.log(a.value)
```

```
function strFun(p1) {
    p1.value = "Eleven"
    p1.index++
}
var a = { value: "Ten", index: 0 }
strFun(a)
console.log(a.index)
```

Visit this link for the detailed discussion:

https://stackoverflow.com/questions/13104494/does-javascript-pass-by-reference

JS Functions – Function Expression

- Functions can also be created by a function expression
- Such a function can be anonymous; it does not have to have a name. For example, the function square could have been defined as:

```
const square = function(number) { return number * number }
var x = square(4) // x gets the value 16
```

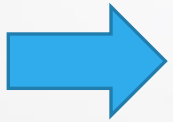
- However, a name can be provided with a function expression. Providing a name allows the function to refer to itself

```
const factorial = function fac(n) { return n < 2 ? 1 : n * fac(n - 1) }
console.log(factorial(3))
```

JS Arrow Functions

- Arrow functions were introduced in ES6
- An arrow function expression has a shorter syntax compared to function expressions

```
hello = function () {
    return "Hello World!";
}
console.log(hello())
```



```
hello = () => {
    return "Hello World!";
}
console.log(hello())
```

- It gets shorter! If the function has only one statement, and the statement returns a value
- Arrow Functions Return Value by Default

```
hello = () => "Hello World!";
console.log(hello())
```

JS Arrow Function with Parameters

```
hello = (val) => "Hi " + val;
console.log(hello(5))
```

- If you have parameters, you pass them inside the parentheses

```
hello = val => "Hi " + val;
console.log(hello(5))
```

- If you have only one parameter, you can skip the parentheses as well

What is Expected Output?

```
const mult = (a, b) => a = a !== undefined ? a : 0; b = b !== undefined ? b : 1; a * b;
console.log(mult(2, 3))
```

```
const mult = () => {
    a = a !== undefined ? a : 0;
    b = b !== undefined ? b : 1;
    a * b;
}
console.log(mult(2, 3))
```

```
const mult = () => {
    a = a !== undefined ? a : 0;
    b = b !== undefined ? b : 1;
    return a * b;
}
console.log(mult(2, 3))
```

```
const mult = () => {
    var a = a !== undefined ? a : 0;
    var b = b !== undefined ? b : 1;
    return a * b;
}
console.log(mult(2, 3))
```

```
const mult = (a, b) => {
    a = a !== undefined ? a : 0;
    b = b !== undefined ? b : 1;
    return a * b;
}
console.log(mult(2, 3))
```

References

- MDN Web Docs: Details of the object model (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Details_of_the_Object_Model)
- W3Schools React ES6 (https://www.w3schools.com/react/react_es6.asp)